

実環境を見据えた 音による行動認識モデルの精度向上

学籍番号： hoge

氏名： hoge

指導教員名： hugahuga 教授

2026 年 6 月 26 日

論文要旨

少子高齢化が急速に進行する日本において、介護資源の不足は深刻な社会問題であり、ICT を活用して高齢者の自立生活を支援する Ambient Assisted Living (AAL) の重要性が高まっている。...

目次

1.	はじめに	2
2.	提案手法	2
2.1	DCASE 基礎モデル	2
2.1.1	音響イベント検出 (SED)	2
3.	評価実験	4
3.1	実験内容	4
4.	実験結果と考察	5
5.	今後の展望	6
	参考文献	7
A.	付録	8
A.1	概要	8
A.2	CMT	8
A.2.1	初期化	8
A.2.2	擬似ラベル作成	9
A.2.3	信頼度重み算出	10
A.2.4	信頼度重みによる損失算出	13

1. はじめに

日本における少子高齢化の進行に伴い、医療・介護資源の不足は喫緊の課題である。この解決策の一つとして、情報技術を活用して高齢者や障がい者の生活支援を行う Ambient Assisted Living (AAL) が挙げられる。AAL は在宅での自立生活を促進することで、医療や介護で発生する労働時間や社会保障費などの社会的コストを抑制すると同時に、被介護者の生活の質 (Quality of Life; QoL) 向上に寄与することを目指すものである [1]。

... 本論文は次のように構成される。2 節では本研究が参考に行っている実装を簡単に紹介し、提案手法について述べる。3 節では実験で用いられる指標と、実験内容を述べる。4 節は実験の結果を整理し、考察を行う。5 節は本論文の成果をまとめ、今後の課題を述べる。

2. 提案手法

2.1 DCASE 基礎モデル

2.1.1 音響イベント検出 (SED)

DCASE 2024 では、タスクとして SED を扱う。

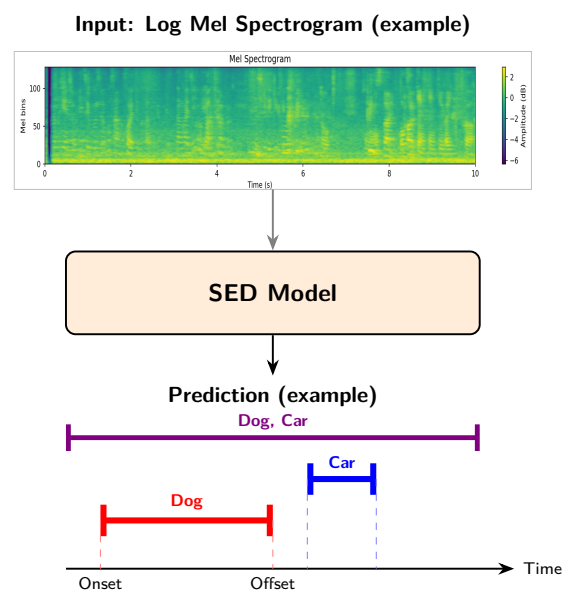


図 2.1 クリップ予測及びフレーム予測の、10 秒の音声入力を対象とした出力例。全体におけるイベントの有無を判定するクリップ予測に対し、フレーム予測ではイベントの区間（Onset/Offset）も算出する。注： 入力のスペクトログラムと予測結果はあくまで構造を説明するためのイメージであり、実際の対応はない。

本タスクでは、10 秒間の音声データを「クリップ」、それをさらに短時間で分割した単位を「フレーム」と定義し、イベント種別及び各イベントの開始・終了時刻を予測する。音声データ内（クリップ及びフレーム）では、複数のイベントが重複して発生する可能性があるため、SED では複数のイベント種別とその発生時刻を同時に予測することが求められる。図 2.1 に示すように、学習においてはクリップ単位では 10 秒間にどのイベントが含まれているか（イベントの有無）を、フレーム単位ではイベント種別に加えて、その開始・終了時刻（時間的境界）を学習対象とする。

また、予測の粒度に応じてデータのラベル定義が異なる。時間的な詳細情報を含むものを「強ラベル」、イベントの有無のみを示すものを「弱ラベル」とし、それぞれフレームとクリップに対応する。

3. 評価実験

3.1 実験内容

ベースラインに対する精度の向上幅及び各要素の有効性を示すため、本研究では 3 つの実験を行う。

まず、手法間の比較を行う。ベースライン (Mean Teacher)、先行研究 (CMT)、及び提案手法の派生形である CMT_{neg} と CMT_{scale} を対象とし、それぞれの識別性能を評価する。

次に、アーキテクチャの検討を行う。MixStyle を既存モデルに直接適用する場合と、軽量 CNN を介して統合する場合とで、精度の変化を比較する。

最後に、構成要素の比較を行う。主要な 3 つのモデル (Mean Teacher, CMT, CMT_{scale}) に対し、MixStyle のみ・SEBBs のみ・両方を適用した条件で実験を行い、各モジュールの導入効果を検証する。

データセットとして、DESED と MAESTRO Real を使用する。

また、実験環境は以下の通り：

- OS: Ubuntu 24.04.3 LTS (Noble Numbat)
- Kernel: 6.14.0-36-generic
- CPU: AMD Ryzen 7 5700X 8-Core Processor, x86_64

- GPU: NVIDIA GeForce RTX 4070, 12282 MiB
- RAM: 15GiB

4. 実験結果と考察

実験結果について整理する。

Model	PSDS1	PSDS2	mpAUC
Mean Teacher(MT)	0.369	0.581	0.720
Confident Mean Teacher(CMT)	0.297	0.60	0.689
CMT_{neg}	0.369	0.543	0.661
CMT_{scale}	0.368	0.621	0.699

表 4.1 Mean Teacher 比較

表 4.1 に各手法の精度比較を示す。Mean Teacher (MT) と比較して、CMT は PSDS2 が向上した一方で、PSDS1 及び mpAUC は低下した。これに対し、 CMT_{neg} では CMT で見られた PSDS1 の低下は改善されたものの、その他の指標においては精度が悪化した。 CMT_{scale} は、MT と同等の PSDS1 を維持しつつ、PSDS2 においては CMT を上回る性能を示した。ただし、mpAUC に関しては MT には及ばなかった。

考察として、CMT における PSDS1 の低下は、非イベント区間の情報（負例）が欠落したことで、時間的な予測が不安定になったことに起因すると考えられる。 CMT_{neg} では負例を導入することでこの問題を解消したが、正例に対して負例の割合が過多となり、逆にクラス分類（PSDS2 など）の予測が不安定化した。以上の結果から、 CMT_{scale} における正例と負例の量的バランスの調整が、時間精度とクラス分類精度の双方を安定化させるために有効であったと結論づけられる。

5. 今後の展望

本研究では、DCASE2024 Task 4 のベースラインモデルを基盤とし、半教師あり学習手法 Mean Teacher (MT) の改良、ドメイン汎化、及び後処理の改善を統合することで、音響イベント検出の精度向上を図った。結果として、負例を活用した CMT (CMT_{scale}) は、従来の MT や単純な CMT と比較して精度向上を達成した。また、周波数ごとの重要度に基づく MixStyle の統合が有効であることを再確認し、軽量の CNN を用いた場合でもドメイン汎化の効果が得られることを示した。

参考文献

- [1] S. Blackman, C. Matlo, C. Bobrovitskiy, A. Waldoch, M. L. Fang, P. Jackson, A. Mihailidis, L. Nygård, A. Astell, and A. Sixsmith. Ambient assisted living technologies for aging well: a scoping review. *Journal of Intelligent Systems* 25.1 (2016), pp. 55–69.
- [2] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. Luk, B. Maher, Y. Pan, C. Puhersch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, M. Suo, P. Tillet, E. Wang, X. Wang, W. Wen, S. Zhang, X. Zhao, K. Zhou, R. Zou, A. Mathews, G. Chanan, P. Wu, and S. Chintala. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, 2024. doi: 10.1145/3620665.3640366. URL: <https://docs.pytorch.org/assets/pytorch2-2.pdf>.
- [3] J. Hwang, M. Hira, C. Chen, X. Zhang, Z. Ni, G. Sun, P. Ma, R. Huang, V. Pratap, Y. Zhang, A. Kumar, C.-Y. Yu, C. Zhu, C. Liu, J. Kahn, M. Ravanelli, P. Sun, S. Watanabe, Y. Shi, Y. Tao, R. Scheibler, S. Cornell, S. Kim, and S. Petridis. TorchAudio 2.1: Advancing speech recognition, self-supervised learning, and audio processing components for PyTorch. 2023. arXiv: 2310.17864 [eess.AS].
- [4] W. Falcon and The PyTorch Lightning team. *PyTorch Lightning*. Version 1.4. 2019. doi: 10.5281/zenodo.3828935. URL: <https://github.com/Lightning-AI/lightning>.

A. 付録

A.1 概要

ソースコードは主に訓練から評価まで行う `train_pretrained.py` と、モデルのあらゆる定義を行う `sed_trainer_pretrained.py` から成る。付録では、CMT と MixStyle の主要な実装を示す。なお、SEBBs は <https://github.com/merlresearch/sebbs> の実装に変更を加えず、呼び出す形で使用しているため、省略する。その他、ソースコードは https://github.com/custard-1855/dcase_base にて公開している。

また、使用したライブラリは以下の通り。

- 深層学習フレームワーク: PyTorch[2]
- 音声前処理: torchaudio[3]
- モデル実装: PyTorch Lightning[4] ...

A.2 CMT

A.2.1 初期化

```
1 # sed_trainer_pretrained.py の__init__内で初期化
2 self.cmt_enabled: bool = self.hparams.get("cmt", {}).get("enabled", False)
3 self.phi_clip: float = float(self.hparams.get("cmt", {}).get("phi_clip", 0.5))
4 self.phi_frame: float = float(self.hparams.get("cmt", {}).get("phi_frame", 0.5))
5 self.phi_neg: float = float(self.hparams.get("cmt", {}).get("phi_neg", 0.3))
6 self.phi_pos: float = float(self.hparams.get("cmt", {}).get("phi_pos", 0.7))
7 self.pos_neg_scale: bool = self.hparams.get("cmt", {}).get("pos_neg_scale", False)
8 self.cmt_warmup_epochs: int = int(self.hparams.get("cmt", {}).get("warmup_epochs", 50))
9 self.use_neg_sample: bool = self.hparams.get("cmt", {}).get("use_neg_sample", False)
```

A.2.2 擬似ラベル作成

```
1 def apply_cmt_postprocessing(  
2     self,  
3     y_w: torch.Tensor,  
4     y_s: torch.Tensor,  
5     index_weak: int,  
6 ) -> tuple[torch.Tensor, torch.Tensor]:  
7     """Apply Confident Mean Teacher postprocessing only unlabeled data.  
8  
9     y_w: (batch, classes)  
10    y_s: (batch, classes, frames)  
11    """  
12    y_w_unlabeled = y_w[index_weak:] # (unlabeled_batch, classes)  
13    y_s_unlabeled = y_s[index_weak:] # (unlabeled_batch, classes, frames)  
14  
15    if self.use_neg_sample:  
16        # Weak (clip-level)  
17        y_tilde_w = torch.where(  
18            y_w_unlabeled >= self.phi_pos,  
19            torch.ones_like(y_w_unlabeled), # 正例 → 1.0  
20            torch.where(  
21                y_w_unlabeled <= self.phi_neg,  
22                torch.zeros_like(y_w_unlabeled), # 負例 → 0.0  
23                y_w_unlabeled, # 中間値 → そのまま (後で信頼度 0)  
24            ),  
25        )  
26  
27    # Strong (frame-level) - Weak 制約なし、独立判定  
28    y_s_binary = torch.where(  
29        y_s_unlabeled >= self.phi_pos,  
30        torch.ones_like(y_s_unlabeled), # 正例 → 1.0  
31        torch.where(  
32            y_s_unlabeled <= self.phi_neg,  
33            torch.zeros_like(y_s_unlabeled), # 負例 → 0.0  
34            y_s_unlabeled, # 中間値 → そのまま (後で信頼度 0)  
35        ),  
36    )  
37
```

```

38     else:
39         y_tilde_w = (y_w_unlabeled > self.phi_clip).float() # clip
40         y_w_expanded = y_tilde_w.unsqueeze(-1).expand_as(y_s_unlabeled)
41         y_s_binary = y_w_expanded * (y_s_unlabeled > self.phi_frame).float() # frame
42
43     y_tilde_s_list = []
44     original_device = y_s.device
45     y_s_numpy = y_s_binary.detach().cpu().numpy() # y_s_binary: (batch, classes,
46         ↪ frames)
47
48     for i in range(y_s_unlabeled.shape[0]):
49         sample = y_s_numpy[i].transpose(1, 0) # (classes, frames) -> (frames, classes)
50         filtered = self.median_filter(sample)
51         y_tilde_s_list.append(filtered)
52
53     y_tilde_s_np = np.stack(y_tilde_s_list, axis=0) # (batch, frames, classes)
54     y_tilde_s_tensor = torch.from_numpy(y_tilde_s_np).to(original_device)
55     y_tilde_s = y_tilde_s_tensor.transpose(1, 2) # -> (batch, classes, frames)
56
57     y_tilde_w = torch.cat((y_w[:index_weak], y_tilde_w), dim=0)
58     y_tilde_s = torch.cat((y_s[:index_weak], y_tilde_s), dim=0)
59
60     return y_tilde_w, y_tilde_s

```

A.2.3 信頼度重み算出

```

1 def compute_cmt_confidence_weights(
2     self,
3     y_w: torch.Tensor,
4     y_s: torch.Tensor,
5     y_tilde_w: torch.Tensor,
6     y_tilde_s: torch.Tensor,
7     index_weak: int,
8 ) -> tuple[torch.Tensor, torch.Tensor]:
9     """Compute confidence weights based on teacher's certainty.
10
11     Correction: High confidence for both positive (near 1) and negative (near 0)
12     ↪ predictions.
13     """

```

```

13     if self.use_neg_sample:
14         # --- use neg_sample ---
15         # Unlabeled 部分のマスク生成
16         y_w_unlabeled = y_w[index_weak:]
17         y_s_unlabeled = y_s[index_weak:]
18
19         pos_mask_w_unlabeled = (y_tilde_w[index_weak:] == 1.0).float()
20         neg_mask_w_unlabeled = (y_tilde_w[index_weak:] == 0.0).float()
21
22         pos_mask_s_unlabeled = (y_tilde_s[index_weak:] == 1.0).float()
23         neg_mask_s_unlabeled = (y_tilde_s[index_weak:] == 0.0).float()
24
25         # Unlabeled 部分の信頼度計算
26         c_w_pos_unlabeled = pos_mask_w_unlabeled * y_w_unlabeled
27         c_w_neg_unlabeled = neg_mask_w_unlabeled * (1.0 - y_w_unlabeled)
28         c_w_unlabeled = c_w_pos_unlabeled + c_w_neg_unlabeled
29
30         c_s_pos_unlabeled = pos_mask_s_unlabeled * y_s_unlabeled
31         c_s_neg_unlabeled = neg_mask_s_unlabeled * (1.0 - y_s_unlabeled)
32         c_s_unlabeled = c_s_pos_unlabeled + c_s_neg_unlabeled
33
34         # Labeled 部分は常に信頼度 1.0
35         c_w_labeled = torch.ones_like(y_w[:index_weak])
36         c_s_labeled = torch.ones_like(y_s[:index_weak])
37
38         # 結合 (pos/neg は後でスケーリングに使用)
39         c_w_pos = torch.cat((c_w_labeled, c_w_pos_unlabeled), dim=0)
40         c_w_neg = torch.cat((torch.zeros_like(y_w[:index_weak]), c_w_neg_unlabeled),
41                               ↪ dim=0)
42         c_w = torch.cat((c_w_labeled, c_w_unlabeled), dim=0)
43
44         c_s_pos = torch.cat((c_s_labeled, c_s_pos_unlabeled), dim=0)
45         c_s_neg = torch.cat((torch.zeros_like(y_s[:index_weak]), c_s_neg_unlabeled),
46                               ↪ dim=0)
47         c_s = torch.cat((c_s_labeled, c_s_unlabeled), dim=0)
48
49     else:
50         # --- CMT ---
51         # Unlabeled 部分のみ計算 (labeled 部分は y_tilde が元の y_w なので除外)
52         y_w_unlabeled = y_w[index_weak:]
53         y_s_unlabeled = y_s[index_weak:]

```

```

52     y_tilde_w_unlabeled = y_tilde_w[index_weak:]
53     y_tilde_s_unlabeled = y_tilde_s[index_weak:]
54
55     c_w_unlabeled = y_w_unlabeled * y_tilde_w_unlabeled
56     y_w_expanded = y_w_unlabeled.unsqueeze(-1).expand_as(y_s_unlabeled)
57     c_s_unlabeled = y_s_unlabeled * y_w_expanded * y_tilde_s_unlabeled
58
59     # Labeled 部分は常に信頼度 1.0
60     c_w_labeled = torch.ones_like(y_w[:index_weak])
61     c_s_labeled = torch.ones_like(y_s[:index_weak])
62
63     # 結合
64     c_w = torch.cat((c_w_labeled, c_w_unlabeled), dim=0)
65     c_s = torch.cat((c_s_labeled, c_s_unlabeled), dim=0)
66
67     if self.use_neg_sample and self.pos_neg_scale: # 正例と負例の量を調整
68         # Unlabeled 部分のみの正例と負例の総重みを計算
69         pos_sum_w_unlabeled = c_w_pos_unlabeled.sum().clamp(min=1e-6)
70         neg_sum_w_unlabeled = c_w_neg_unlabeled.sum().clamp(min=1e-6)
71
72         pos_sum_s_unlabeled = c_s_pos_unlabeled.sum().clamp(min=1e-6)
73         neg_sum_s_unlabeled = c_s_neg_unlabeled.sum().clamp(min=1e-6)
74
75         # Unlabeled 部分のみをスケーリング
76         if pos_sum_w_unlabeled < neg_sum_w_unlabeled:
77             # 負例が多い → 負例を縮小
78             scale_factor = neg_sum_w_unlabeled / pos_sum_w_unlabeled
79             c_w_unlabeled_scaled = c_w_pos_unlabeled + c_w_neg_unlabeled / scale_factor
80         else:
81             # 正例が多い → 正例を縮小
82             scale_factor = pos_sum_w_unlabeled / neg_sum_w_unlabeled
83             c_w_unlabeled_scaled = c_w_pos_unlabeled / scale_factor + c_w_neg_unlabeled
84
85         if pos_sum_s_unlabeled < neg_sum_s_unlabeled:
86             scale_factor = neg_sum_s_unlabeled / pos_sum_s_unlabeled
87             c_s_unlabeled_scaled = c_s_pos_unlabeled + c_s_neg_unlabeled / scale_factor
88         else:
89             scale_factor = pos_sum_s_unlabeled / neg_sum_s_unlabeled
90             c_s_unlabeled_scaled = c_s_pos_unlabeled / scale_factor + c_s_neg_unlabeled
91
92     # Labeled とスケール済み Unlabeled を結合

```

```

93     c_w = torch.cat((c_w_labeled, c_w_unlabeled_scaled), dim=0)
94     c_s = torch.cat((c_s_labeled, c_s_unlabeled_scaled), dim=0)
95
96     # ログを更新（スケール前の値を記録）
97     self.log("train/cmt/pos_neg_ratio_weak", pos_sum_w_unlabeled /
98             ↪ neg_sum_w_unlabeled)
99     self.log("train/cmt/pos_neg_ratio_strong", pos_sum_s_unlabeled /
100            ↪ neg_sum_s_unlabeled)
101
102     return c_w, c_s

```

A.2.4 信頼度重みによる損失算出

```

1 def compute_cmt_consistency_loss(
2     self,
3     student_w: torch.Tensor,
4     student_s: torch.Tensor,
5     teacher_pseudo_w: torch.Tensor,
6     teacher_pseudo_s: torch.Tensor,
7     confidence_w: torch.Tensor,
8     confidence_s: torch.Tensor,
9 ) -> tuple[torch.Tensor, torch.Tensor]:
10     """Compute weighted BCE loss."""
11     # Weak Loss
12     bce_w = F.binary_cross_entropy(student_w, teacher_pseudo_w, reduction="none")
13     weighted_bce_w = confidence_w * bce_w
14     loss_w_con = weighted_bce_w.mean()
15
16     # Strong Loss
17     bce_s = F.binary_cross_entropy(student_s, teacher_pseudo_s, reduction="none")
18     weighted_bce_s = confidence_s * bce_s
19     loss_s_con = weighted_bce_s.mean()
20
21     self.log("train/cmt/confidence_weak_mean", confidence_w.mean())
22     self.log("train/cmt/confidence_weak_std", confidence_w.std())
23     self.log("train/cmt/confidence_strong_mean", confidence_s.mean())
24     self.log("train/cmt/confidence_strong_std", confidence_s.std())
25
26     return loss_w_con, loss_s_con

```
